VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY (HCMUT)
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Course: **CO2001 – Professional Skills for Engineers**
Class CC02 – Group 13 – Semester 1, Academic Year 2026–2027

HCMUT-AILib

A Zero-Resource Semantic Retrieval and Research Assistant for Digital Academic Libraries

Milestone 1 – Project Outline Report (Week 3)

Member	Role
Nguyễn Phước Bảo Phú	Project Lead & System Architect
Từ Công Phú	Data & Retrieval Engineer
Nguyễn Vi Kiên	Frontend & Integration Engineer
Trần Kiện Đông	QA, Ethics & Compliance Lead

Repository: `github.com/HCMUT-Fus/hcmut-ailib` · Licence: GNU GPLv3

Ho Chi Minh City, 2026

Table of Contents

Abstract
1. Introduction
    1.1 Objectives
    1.2 Scope
2. Step 1 – Identify the Problem
3. Step 2 – Criteria and Constraints
    3.1 Constraints
    3.2 Criteria (measurable)
4. Step 3 – Brainstorm Possible Solutions
5. Step 4 – Generate Ideas
6. Step 5 – Explore Options (Feasibility Decision Matrix)
7. Step 6 – Select an Approach
    7.1 System architecture
    7.2 Interfaces between modules
8. User Requirements Analysis and Planned User Interface
    8.1 Method
    8.2 Personas
    8.3 User stories and functional requirements
    8.4 Non-functional requirements
    8.5 Planned interface
9. Work Breakdown Structure (8 Weeks)
10. Responsibility Assignment (RACI)
11. Risk Register
12. Team Governance
13. Expected Outcomes and Evaluation Plan
14. Conclusion
References

Abstract

University library catalogues (OPAC) still match the exact words a user types against titles and subject headings. When a student describes a research need in natural language ("how to translate between languages that have little training data"), the catalogue fails whenever the relevant papers use different vocabulary ("low-resource neural machine translation"). This is the *vocabulary mismatch problem* [1]. HCMUT-AILib is a local-first, zero-cost semantic search engine and research assistant for academic papers. It ingests open metadata for 1,000 computer-science papers from the public arXiv API, encodes each title and abstract into a 384-dimensional sentence embedding with the open `all-MiniLM-L6-v2` model [2], [3], [13], stores the vectors in an embedded ChromaDB index [11], and serves ranked results through a Streamlit web interface [12], all on a standard student laptop without a GPU or any paid cloud service. This report covers the first six steps of the engineering design process taught in CO2001: problem identification, criteria and constraints, brainstorming, idea generation, option exploration with a weighted feasibility matrix, and selection of the final approach. It also presents the four-layer architecture, the user requirements and planned interface, the 8-week work breakdown, the responsibility (RACI) matrix, the risk register and the team's governance rules. The selected design (Option C, local dense vector retrieval) scores 4.40/5 in the weighted decision matrix, against 3.85 for a local inverted index and 2.90 for a cloud LLM. The measurable targets are a query latency below 1.5 s and a Mean Reciprocal Rank (MRR) above 0.7 on a 30-query labelled benchmark.

1. Introduction

Academic libraries increasingly serve students who search the way they use web search engines: they type a question or a description of a concept rather than a controlled-vocabulary term. Lexical systems, both the Boolean OPAC and ranked keyword engines such as BM25 [4], score a document only by the query words it contains. As a result they miss synonyms ("car" and "automobile"), paraphrases, and cross-disciplinary terminology. Dense retrieval with pre-trained sentence encoders [2], [8] maps queries and documents into a shared vector space where semantically related texts lie close together even if they share no words.

The CO2001 course asks teams to run a complete, professionally managed engineering project: design-process reasoning, teamwork and governance, ethical and legal analysis, and an evaluated prototype. HCMUT-AILib was scoped to meet these requirements with zero financial cost and zero physical resources. It uses only open metadata, open-source software and the team's own laptops.

1.1 Objectives

1. **O1:** Build a reproducible pipeline that ingests 1,000 open-access paper records from the arXiv API while respecting its terms of use [10].

2. **O2:** Index the records as dense vectors and answer natural-language queries in under 1.5 s on a CPU-only laptop.
3. **O3:** Reach MRR > 0.7 and report Precision@5 on a 30-query benchmark with human-labelled ground truth, and compare against a lexical BM25 baseline.
4. **O4:** Provide research-assistant features: metadata filters, "find similar papers", extractive keywords and ready-to-paste citations (BibTeX, APA, IEEE).
5. **O5:** Run the project under professional governance (Git flow, code review, CI, weekly meetings, RACI) and complete an ethics and legal audit (Milestone 2).

1.2 Scope

In scope: metadata and abstracts of 1,000 arXiv papers in four categories (cs.AI, cs.LG, cs.CL, cs.IR) submitted in 2021–2025, 50 papers per category per year; English-language queries; single-user local deployment. *Out of scope:* full-text PDF indexing or redistribution, user accounts, generative answer synthesis, multilingual retrieval, and integration with the HCMUT library's production OPAC.

2. Step 1 – Identify the Problem

Problem statement. HCMUT students and researchers lose time and miss relevant literature because catalogue search requires them to guess the exact terms authors used. Furnas et al. showed that two people choose the same term for a concept with probability below 0.2 [1]. Keyword search therefore fails systematically for conceptual, paraphrased and interdisciplinary queries.

Who is affected. Undergraduate students who are new to a field's jargon (the largest group), graduate researchers doing literature reviews, and librarians who support reference queries.

Evidence and framing. We framed the need with a *5W1H* analysis during the Week 1 problem-definition session:

Question	Answer
What	Natural-language queries fail to retrieve relevant papers in keyword catalogues
Who	Students, researchers, librarians at HCMUT
Where	Digital library and OPAC search interfaces
When	Literature search at the start of theses, capstone projects and course assignments
Why	Exact string matching ignores synonymy and conceptual similarity [1], [7]
How (to solve)	Retrieve by meaning: embed queries and documents in a shared semantic vector space

Need statement. *"Students need a way to find academic papers by describing a concept in their own words, so that relevant work is found even when it uses different terminology, at no cost and without sending their queries to third parties."*

3. Step 2 – Criteria and Constraints

Constraints are hard limits that any acceptable design must satisfy. Criteria are measurable qualities used to compare acceptable designs.

3.1 Constraints

ID	Constraint	Rationale
C1	**Zero cost ($0.00):** no paid APIs or cloud services	Student project without funding
C2	**Commodity hardware:** 4-core CPU, 8 GB RAM, no discrete GPU	Must run on any team member's or user's laptop
C3	**Zero physical goods:** no scanning, barcodes or specialised devices	Course scoping requirement
C4	**Open data only:** metadata from public APIs; no full-text redistribution	Copyright and licence compliance
C5	**8-week schedule, 4 members**	CO2001 timeline
C6	**Open-source licence:** distributed under GNU GPLv3 [15]	Team decision; transparency and reuse
C7	**Legal compliance:** Vietnamese IT, IP and personal-data law; arXiv API Terms of Use	Professional responsibility; detailed in Milestone 2

3.2 Criteria (measurable)

ID	Criterion	Target	Measurement
K1	Query latency	< 1.5 s per query (warm cache) on 1,000 records	Timed end-to-end `search()` calls, median and maximum over the benchmark
K2	Retrieval accuracy	MRR > 0.7	30 natural-language queries with manually labelled relevant papers; MRR as defined for TREC-8 QA [5]
K3	Early precision	Precision@5 reported and compared with BM25	Same benchmark
K4	Indexing time	< 3 min to embed 1,000 records on CPU	Timed `indexer.py` run
K5	Usability	SUS score ≥ 68 (industry average)	System Usability Scale [14] with 10–15 HCMUT students
K6	Reproducibility	Fresh clone to running app in 3 commands	Reproducibility test on a clean machine
K7	Quality assurance	100 % of automated tests pass in CI	GitHub Actions on every pull request

MRR is the mean, over all queries Q, of the reciprocal rank of the first relevant result: MRR = $(1/|Q|)\ \Sigma\ 1/\text{rank}_q$, with 1/rank = 0 when no relevant paper is returned in the top k.

4. Step 3 – Brainstorm Possible Solutions

During Week 2 the team used a **Think–Pair–Share (TPS)** protocol. Each member first generated solutions individually (Think, 10 min). Members then compared and merged ideas in two pairs (Pair, 15 min). Finally, each pair presented to the whole team, and ideas were clustered on a shared concept map (Share, 20 min). The solution space is summarised in `docs/mindmaps/design_options.md`. Three families of solutions emerged:

- **Option A – Cloud generative LLM.** Send queries (and possibly documents) to a hosted LLM API (e.g., OpenAI or Anthropic) to rank papers or generate answers with retrieval-augmented generation.
- **Option B – Local inverted index.** Full-text keyword search with BM25 ranking using Apache Lucene, Elasticsearch, or a lightweight Python BM25 implementation [4], [7].
- **Option C – Local dense vector retrieval.** Encode titles and abstracts with a compact sentence-transformer [2], [3]; store vectors in an embedded vector database; rank by cosine similarity with approximate nearest-neighbour search (HNSW) [6].

5. Step 4 – Generate Ideas

Each option was refined into candidate features and improvements. Each idea was then tagged by value and effort:

Idea	Description	Decision
I1 Hybrid search	Combine BM25 lexical scores with vector similarity	Deferred to future work; BM25 built **as a baseline** for the A/B comparison
I2 Abstract summariser	Quantised local LLM summarising abstracts on CPU	Rejected: too slow on C2 hardware and adds hallucination risk; replaced by **extractive TF-IDF keywords**
I3 Auto-classification	Classify papers into ACM CCS or Dewey classes	Deferred; arXiv categories already provide a usable taxonomy
I4 Find similar papers	Use a paper's own embedding as the query	**Accepted:** low effort, high research value
I5 Citation export	One-click BibTeX/APA/IEEE	**Accepted:** directly supports academic integrity
I6 Metadata filters	Year range, category, author	**Accepted:** standard in academic search tools
I7 Normalised score	Show relevance as a 0–100 % match	**Accepted:** improves transparency

6. Step 5 – Explore Options (Feasibility Decision Matrix)

The three options were compared with a weighted decision matrix (full version in `docs/feasibility_matrix.md`). The weights were agreed by consensus before scoring to avoid post-hoc bias. Each member scored independently on a 1–5 scale (5 = best), and the scores below are the agreed consensus values.

Criterion	Weight	A: Cloud LLM	B: Inverted Index	C: Local Vector DB
Financial cost (C1)	0.25	1	5	5
Semantic retrieval quality (O3)	0.25	5	2	4
Hardware fit, 8 GB CPU laptop (C2)	0.15	5	3	4
Data privacy & offline operation (C7)	0.15	1	5	5
Query latency (K1)	0.10	2	5	4
Engineering feasibility in 8 weeks (C5)	0.10	3	4	4
Weighted total (max 5.00)	**1.00**	**2.90**	**3.85**	**4.40**

Score rationale (abridged): A has recurring token costs that violate C1, sends user queries to a vendor, and depends on network round-trips (often several seconds), although its semantic quality is the highest. B is free, fast and offline, but it matches only literal terms, so it does not address the core vocabulary-mismatch problem. Elasticsearch also needs a JVM with a sizeable heap. C is free and offline, captures semantic similarity, and runs on a CPU: the model has about 22 M parameters and roughly 90 MB of weights [13]. Its main costs are a one-off embedding step and a slightly higher per-query latency than B, because each query must be encoded.

Sensitivity check: with equal weights (1/6 each), the totals are A = 2.83, B = 4.00 and C = 4.33. C remains the best option in every weighting we tested (`docs/feasibility_matrix.md`, Section 5). B would overtake C only if latency outweighed twice the semantic-quality weight plus the hardware weight, which would contradict the problem statement.

7. Step 6 – Select an Approach

Option C, local dense vector retrieval, is selected. The concrete technology choices are:

Component	Choice	Justification
Data source	arXiv API (Atom feed), stdlib `urllib`, 3 s delay between requests	Free, open metadata (CC0) [9], [10]
Embedding model	`sentence-transformers/all-MiniLM-L6-v2` (384-d, Apache-2.0)	Strong quality-to-size ratio; fast on CPU [2], [3], [13]

Component	Choice	Justification
Text to embed	`"<title>. <abstract>"`	The title adds a concise topical signal to the abstract
Vector store	ChromaDB `PersistentClient`, cosine space, HNSW index	Embedded (no server process), persistent on disk, Apache-2.0 [6], [11]
Ranking score	cosine similarity, displayed as (1 + cos)/2 ∈ [0, 1]	Interpretable "percentage match"
Lexical baseline	BM25 (Okapi) [4]	Controlled A/B comparison for K2/K3
Interface	Streamlit with cached model and DB handles	Rapid, Python-only UI [12]
Quality	`pytest` + GitHub Actions CI	Automated regression checks (K7)

7.1 System architecture

The system is organised into four layers with one-directional dependencies (Figure 1).

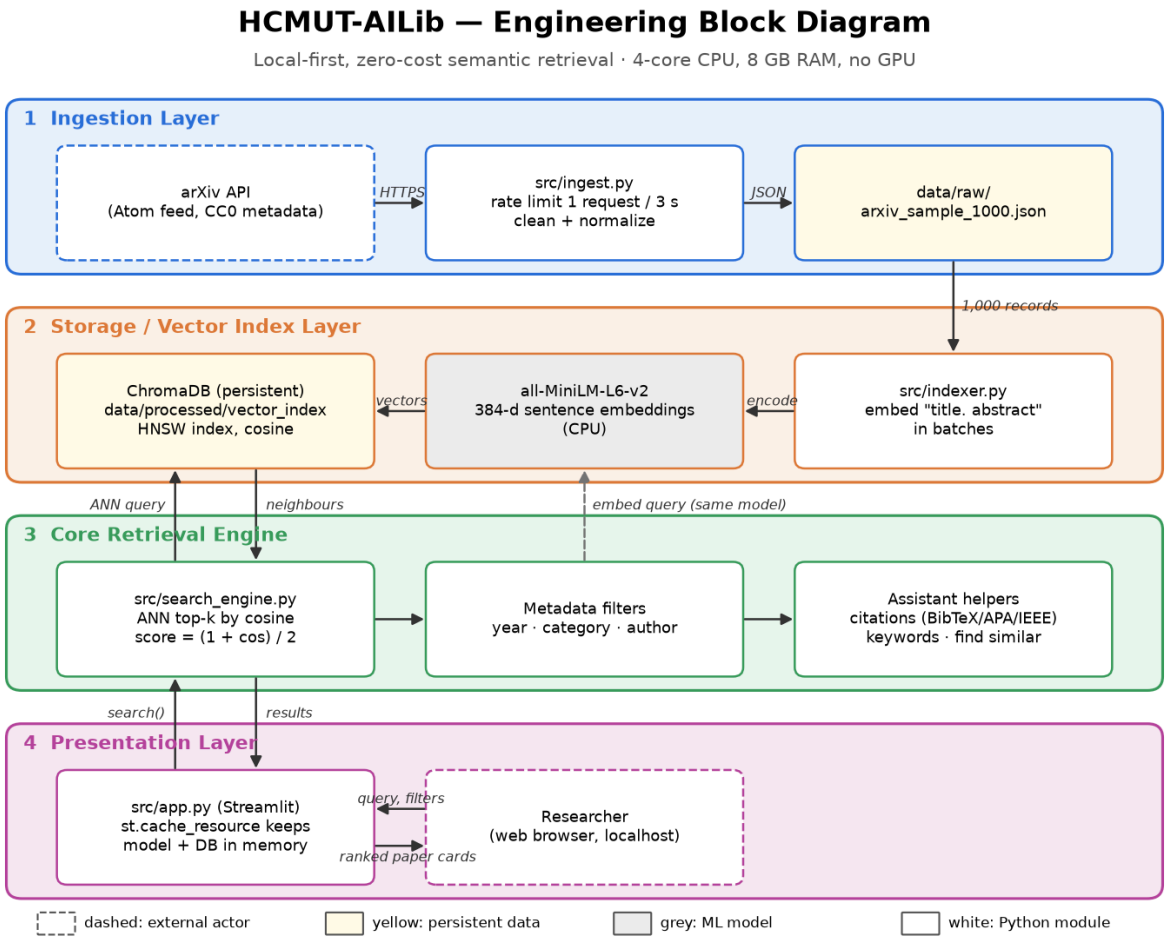


Figure 1. Four-layer architecture: Ingestion → Storage/Vector Index → Core Retrieval Engine → Presentation.

1. **Ingestion Layer (`src/ingest.py`).** Queries the arXiv API by category and submission-date window (cs.AI, cs.LG, cs.CL, cs.IR × 2021–2025 × 50 papers). It respects the 1 request / 3 s limit and normalises records: collapses whitespace, uses ISO 8601 dates, strips arXiv version suffixes from IDs, and sets `doi` to null when absent. Output: `data/raw/arxiv_sample_1000.json` (schema in `docs/data_schema.md`).
2. **Storage / Vector Index Layer (`src/indexer.py`).** Encodes `title. abstract` in batches with `all-MiniLM-L6-v2` and upserts the vectors, documents and metadata (year, categories, authors, DOI, PDF URL) into a ChromaDB persistent collection at `data/processed/vector_index/` in cosine space.
3. **Core Retrieval Engine (`src/search_engine.py`, `src/citations.py`, `src/keywords.py`, `src/lexical_baseline.py`).** Embeds the query, retrieves the top-*k* nearest neighbours with optional metadata filters (year range, category, author), normalises scores, and provides find-similar, citation formatting, TF-IDF keyword extraction and a BM25 baseline for evaluation.
4. **Presentation Layer (`src/app.py`).** A Streamlit application with a search tab, a similar-papers tab and an About tab. Expensive resources (the model and the DB client) are cached with `st.cache_resource`.

Data flows strictly downward at build time (API → JSON → vectors) and upward at query time (UI → engine → index → engine → UI). Each layer can be tested in isolation: the ingestion schema with `tests/test_ingest.py`, retrieval quality with `tests/test_retrieval.py`, and the UI with manual end-to-end checks.

7.2 Interfaces between modules

Producer → Consumer	Contract
`ingest` → `indexer`	JSON list of records with fields `id, title, abstract, authors, doi, categories, primary_category, published_date, pdf_url`
`indexer` → `search_engine`	ChromaDB collection; document ID = arXiv ID; metadata scalars only (lists serialised to strings)
`search_engine` → `app`	`search(query, k, filters) → list[Result]` with paper fields plus `score ∈ [0, 1]`; `find_similar(paper_id, k)`
`citations` / `keywords` → `app`	Pure functions over a record: formatted strings and a keyword list

8. User Requirements Analysis and Planned User Interface

Chapter owner: Nguyễn Vi Kiên (Frontend & Integration Engineer).

8.1 Method

User requirements came from three sources:

1. An informal review of established academic search interfaces (Semantic Scholar, arXiv listing and search pages, Google Scholar), focusing on result-card content and filter placement.
2. Personas derived from the team's own experience as HCMUT students.
3. The feature concept map produced during brainstorming (`docs/mindmaps/feature_requirements.md`).

The requirements will be validated in Week 7 through usability testing with 10–15 HCMUT students using the System Usability Scale [14] (protocol in `docs/usability/sus_survey.md`).

8.2 Personas

Persona	Goal	Pain point with keyword search
Linh, 3rd-year CS undergraduate starting a capstone project	Find 5–10 foundational papers on a topic she can only describe loosely	Does not yet know the field's terminology
Minh, Master's student writing a literature review	Find papers similar to one he already knows, filtered by recent years	Must issue many synonym queries; citation formatting is tedious
Ms. Hoa, subject librarian	Help students quickly at the reference desk	Students' questions do not map to catalogue subject headings

8.3 User stories and functional requirements

ID	User story	Functional requirement	Priority
FR1	As a student, I want to type a question in plain English and get relevant papers	Free-text query box; ranked results by semantic similarity	Must
FR2	As a researcher, I want to control how many results I see	Top-K slider, 1–20 (default 10)	Must
FR3	As a researcher, I want to limit results to recent years or a field	Sidebar filters: year range (2021–2025), arXiv categories (multi-select), author substring	Must
FR4	As a student, I want to judge relevance quickly	Result card: title, authors, year, category badges, match score (%), truncated abstract with "show full abstract" expander	Must
FR5	As a researcher, I want to read the paper	Links to the arXiv abstract page/PDF and to the DOI when available	Must
FR6	As a student, I want to cite correctly	Copyable BibTeX, APA and IEEE citations per paper	Must
FR7	As a researcher, I want papers like the one I found	"Find similar" action and a Similar-papers tab	Should
FR8	As a student, I want the gist without reading the full abstract	5 extractive keywords per paper	Should

ID	User story	Functional requirement	Priority
FR9	As any user, I want to understand how the tool works and what its limits are	About tab: data source, model, score meaning, licences, limitations, arXiv acknowledgement	Must
FR10	As any user, I want clear feedback	Loading spinner; messages for empty queries and empty result sets	Must

8.4 Non-functional requirements

ID	Requirement
NFR1 Performance	< 1.5 s per query after warm-up (K1); first model load is cached
NFR2 Privacy	No query logging, no telemetry from the application, no user accounts; all processing local
NFR3 Usability	SUS ≥ 68; key task (search → open paper → copy citation) in ≤ 3 interactions
NFR4 Compatibility	Works in current Chrome, Firefox and Edge; supports Streamlit light and dark themes
NFR5 Accessibility	Information is not conveyed by colour alone (scores also shown as numbers); readable contrast
NFR6 Portability	Windows, macOS and Linux with Python ≥ 3.10; `pip install -r requirements.txt`

8.5 Planned interface

The planned Streamlit layout (ASCII wireframes in `docs/ui_wireframes.md`) is:

- **Sidebar:** Top-K slider (1–20), year-range slider, category multi-select, author text filter, and a reset button.
- **Main area, "Search" tab:** a central query box with example queries; a results summary line ("10 results in 0.21 s"); and result cards with title, authors, year, category badges, a score bar and percentage, a collapsible abstract, keywords, buttons for PDF, DOI and *Find similar*, and a citation expander with BibTeX/APA/IEEE code blocks that have a built-in copy button.
- **"Similar papers" tab:** select or receive a seed paper and list its nearest neighbours, excluding the paper itself.
- **"About" tab:** a description of the method, the meaning of the score, the dataset composition, limitations and bias notes, licences, and the arXiv acknowledgement ("Thank you to arXiv for use of its open access interoperability.").

9. Work Breakdown Structure (8 Weeks)

Week	Design step / milestone	Key deliverables	Lead
W1	Steps 1–2: problem and constraints; project setup	Repository structure, branch protection, PR/issue templates, `README.md`, base `requirements.txt`, meeting-minutes template, pytest skeleton	Bảo Phú
W2	Steps 3–5: TPS brainstorming, concept maps, feasibility	`docs/mindmaps/`, `docs/feasibility_matrix.md`, `docs/architecture.png`, CI workflow, CI smoke-test plan	Bảo Phú / Kiên / Đông
W3	Step 6: approach selected; **Milestone 1**	**`reports/outline_report.pdf`**, `src/ingest.py`, `data/raw/arxiv_sample_1000.json`, `tests/test_ingest.py`	Bảo Phú
W4	Embedding and indexing; **Milestone 2**	**`reports/ethics_legal_audit.pdf`**, `src/indexer.py`, `data/processed/vector_index/`	Đông / Công Phú
W5	Step 7a: core engine and benchmark	`src/search_engine.py`, normalised scores, citations, 30-query benchmark, `tests/test_retrieval.py`	Bảo Phú / Đông
W6	Step 7b: UI and integration	Complete `src/app.py`, find-similar, keywords, manual cross-browser end-to-end tests	Kiên
W7	Step 8: evaluation and refinement; **Milestone 3**	BM25 vs. semantic A/B comparison, latency/MRR/P@5 tables, SUS study, **`reports/final_evaluation.pdf`**, tag `v1.0.0`	Bảo Phú / Đông
W8	Final defence and release	Pinned dependencies, reproducibility test, demo video, slides, Q&A script, peer assessment, release notes	All

Critical path: W3 dataset → W4 index → W5 engine and benchmark → W7 evaluation. The UI (W6) runs in parallel with W5 against a stub engine, so a delay in the engine does not block interface work.

10. Responsibility Assignment (RACI)

R = Responsible, A = Accountable (exactly one per row), C = Consulted, I = Informed.

Work package / artefact	Bảo Phú	Công Phú	Kiên	Đông
Repository, Kanban board and CI/CD (`.github/`)	**A/R**	I	I	C
Data schema and ingestion (`src/ingest.py`, `data/raw/`)	C	**A/R**	I	C
Vector indexing and ChromaDB (`src/indexer.py`, `data/processed/`)	C	**A/R**	C	I
Semantic search core (`src/search_engine.py`)	**A/R**	C	C	C
Streamlit interface (`src/app.py`)	C	I	**A/R**	C

Work package / artefact	Bảo Phú	Công Phú	Kiên	Đông
Architecture diagram and concept maps (`docs/`)	C	C	**A/R**	I
Test suite and MRR benchmark (`tests/`)	C	C	I	**A/R**
Outline report (W3)	**A/R**	C	C	C
Ethics & legal audit (W4)	C	C	I	**A/R**
Final evaluation report (W7)	**A/R**	C	C	R
README, user guide and demo video	C	I	**A/R**	C
Weekly meeting minutes (`docs/meeting_minutes/`)	I	I	I	**A/R**

11. Risk Register

Likelihood (L) and impact (I) are rated 1–3; exposure = L × I.

#	Risk	L	I	Exp.	Mitigation	Owner
R1	arXiv API throttling or outage during ingestion	2	3	6	3 s delay, retries with back-off, cache raw responses, commit the JSON dataset so later stages never depend on the API	Công Phú
R2	MRR target (> 0.7) not met	2	3	6	Benchmark early (W5); tune the embedded text (title + abstract) and query phrasing; report honestly with the BM25 comparison if the target is missed	Đông
R3	Latency > 1.5 s on low-end laptops	1	2	2	Cache model and DB client; small model; HNSW index; measure on the weakest team laptop	Bảo Phú
R4	Dependency conflicts (PyTorch / ChromaDB versions)	2	2	4	Virtual environment, pinned `requirements.txt`, CI on a clean runner	Bảo Phú
R5	Ground-truth labelling bias (labellers favour the system's own results)	2	2	4	Label relevance before running the system, using metadata and reading; two labellers per query; disagreements resolved by discussion	Đông
R6	Licence or legal non-compliance (redistributing full texts, missing attribution)	1	3	3	Store metadata only and link to arXiv; Milestone 2 audit; arXiv acknowledgement in the UI and README	Đông
R7	Uneven workload or a member unavailable	2	2	4	RACI with backups; weekly stand-ups; Kanban visibility; peer assessment	Bảo Phú
R8	Scope creep (summariser, hybrid search)	2	2	4	Must/Should prioritisation; Lead approves any new feature	Bảo Phú
R9	Merge conflicts close to deadlines	2	1	2	Small PRs, squash merges, feature branches per member	All

12. Team Governance

- **Branching:** `main` (protected, release only), `develop` (integration), `feat/<member>-<feature>` and `fix/<bug>` branches. No direct commits to `main`/`develop`.
- **Commits:** Conventional Commits, `type(scope): imperative summary`, e.g. `feat(ingest): add rate limiting for arxiv api`.
- **Code review:** every pull request is linked to an issue and labelled, needs at least one approval (Đông for tests and quality, Bảo Phú for architecture), and requires a passing CI run. Merges use *squash and merge*.
- **Meetings:** weekly sync every Saturday, 20:00–21:00 (Google Meet/Discord). Each member answers the three stand-up questions (*done / next / blockers*), the team reviews progress against milestones, and records decisions. Đông publishes the minutes in `docs/meeting_minutes/` within 24 h.
- **Decision-making:** consensus first. Unresolved technical disagreements are decided by the accountable owner of the work package, and by the Lead for cross-cutting issues, with the rationale recorded in the minutes.
- **Conflict and contribution:** workload is visible on the Kanban board. A self- and peer-assessment matrix (`docs/peer_assessment.md`) is completed at the end of the project.
- **Quality gate:** a deliverable is "done" when it is merged into `develop`, all tests pass, its documentation is updated, and it is ticked off in `docs/quality_checklist.md`.

13. Expected Outcomes and Evaluation Plan

At the end of Week 7 the team will report:

1. Latency: median and maximum over the 30 benchmark queries, against K1.
2. MRR and Precision@5 for the semantic engine and for the BM25 baseline, against K2/K3.
3. Indexing time and memory footprint.
4. SUS results.
5. Limitations, including corpus size, the English-only model, and abstract-only indexing.

Results that miss a target will be reported as such, together with an analysis of the causes.

14. Conclusion

HCMUT-AILib addresses a real and well-documented weakness of keyword-based library search with an affordable, privacy-preserving design that fits the course constraints. The weighted analysis favours local dense vector retrieval. The four-layer architecture separates concerns so that each team member owns a testable component, and the governance and risk plans give the 8-week schedule a realistic chance of delivering a working, evaluated prototype.

References

[1] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais, "The vocabulary problem in human-system communication," *Commun. ACM*, vol. 30, no. 11, pp. 964–971, Nov. 1987.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

[3] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 5776–5788.

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] E. M. Voorhees, "The TREC-8 question answering track report," in *Proc. 8th Text REtrieval Conf. (TREC-8)*, NIST Special Publication 500-246, Gaithersburg, MD, USA, 1999, pp. 77–82.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[8] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[9] arXiv, "arXiv API User's Manual." [Online]. Available: https://info.arxiv.org/help/api/user-manual.html (accessed Oct. 2026).

[10] arXiv, "Terms of Use for arXiv APIs." [Online]. Available: https://info.arxiv.org/help/api/tou.html (accessed Oct. 2026).

[11] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com/ (accessed Oct. 2026).

[12] Snowflake Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/ (accessed Oct. 2026).

[13] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2 – model card." [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed Oct. 2026).

[14] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[15] Free Software Foundation, "GNU General Public License, version 3," Jun. 29, 2007. [Online]. Available: https://www.gnu.org/licenses/gpl-3.0.html